

```

1  #####WEATHER TASK 4 ###
2  <html>
3      <head>
4          <title>Weather App</title>
5      </head>
6      <body>
7          <h1>Weather App</h1>
8
9          <input type="text" id="city" placeholder="Enter City" />
10         <button onclick="getWeather()">Get Weather</button>
11
12         <h3 id="result"></h3>
13
14         <script>
15             const API_KEY = "4cac606c2a87c0b26481cdb2f3a46648";
16             async function getWeather() {
17                 let city = document.getElementById("city").value;
18
19                 let url = `https://api.openweathermap.org/data/2.5/weather?q=${city}&units=metric&appid=${API_KEY}`;
20
21                 let response = await fetch(url);
22                 let data = await response.json();
23
24                 document.getElementById("result").innerHTML = `
25                     City : ${data.name}<br>
26                     Temperature : ${data.main.temp} °C<br>
27                     Humidity : ${data.main.humidity}%<br>
28                     Weather : ${data.weather[0].description}
29                 `;
30             }
31         </script>
32     </body>
33 </html>
34 #####TASK5 InsertData.java#####
35 import java.sql.*;
36 import java.util.Scanner;
37 public class InsertData {
38     public static void main(String[] args) {
39         String url = "jdbc:mysql://localhost:3306/timr4";
40         String user = "root";
41         String password = "password";
42         try {
43             Connection con = DriverManager.getConnection(url, user, password);
44             System.out.println("Connected to Database");
45             String sql = "INSERT INTO students(id, name, age, email) VALUES (?, ?, ?, ?)";
46             PreparedStatement ps = con.prepareStatement(sql);
47             Scanner sc = new Scanner(System.in);
48             System.out.println("Inserting Data into student table.");
49
50             System.out.print("Enter student ID: ");
51             int sid = sc.nextInt();
52             sc.nextLine();
53             System.out.print("Enter student name: ");
54             String sname = sc.nextLine();
55             System.out.print("Enter student Age: ");
56             int sage = sc.nextInt();
57             sc.nextLine();
58             System.out.print("Enter student Email: ");
59             String semail = sc.nextLine();
60
61             ps.setInt(1, sid);
62             ps.setString(2, sname);
63             ps.setInt(3, sage);
64             ps.setString(4, semail);
65
66             int rowsAffected = ps.executeUpdate();
67             if (rowsAffected > 0) {
68                 System.out.println("Record Inserted Successfully.");
69             } else {
70                 System.out.println("Record insertion failed.");
71             }
72             sc.close();
73         } catch (SQLException e) {
74             System.out.println("Error: " + e);
75         }
76     }
77 }

```

```

1 //DeleteData.java
2 import java.sql.*;
3 import java.util.Scanner;
4 public class DeleteData {
5     public static void main(String[] args) {
6         String url = "jdbc:mysql://localhost:3306/timr4";
7         String user = "root";
8         String password = "password";
9         try {
10             Connection con = DriverManager.getConnection(url, user, password);
11             System.out.println("Connected to Database");
12
13             Scanner sc = new Scanner(System.in);
14
15             System.out.print("Enter student ID to delete: ");
16             int sid = sc.nextInt();
17
18             String sql = "DELETE FROM students WHERE id=?";
19             PreparedStatement ps = con.prepareStatement(sql);
20             ps.setInt(1, sid);
21
22             int rowsAffected = ps.executeUpdate();
23             if (rowsAffected > 0) {
24                 System.out.println("Record Deleted Successfully.");
25             } else {
26                 System.out.println("No record found with ID: " + sid);
27             }
28             sc.close();
29         } catch (SQLException e) {
30             System.out.println("Error: " + e);
31         }
32     }
33 }
34 #####bookstore.xsd
35 <?xml version="1.0" encoding="UTF-8"?>
36
37 <xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
38
39     <xs:element name="bookstore">
40
41         <xs:complexType>
42
43             <xs:sequence>
44
45                 <xs:element name="book" minOccurs="1" maxOccurs="unbounded">
46
47                     <xs:complexType>
48
49                         <xs:sequence>
50
51                             <xs:element name="title" type="xs:string"/>
52
53                             <xs:element name="author" type="xs:string"/>
54
55                             <xs:element name="price" type="xs:decimal"/>
56
57                         </xs:sequence>
58
59                     </xs:complexType>
60
61                 </xs:element>
62
63             </xs:sequence>
64
65         </xs:complexType>
66
67     </xs:element>
68
69 </xs:schema>

```

```
//ReadData.java
```

```
import java.sql.*;

public class ReadData {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/timr4";
        String user = "root";
        String password = "password";
        try {
            Connection con = DriverManager.getConnection(url, user, password);
            System.out.println("Connected to Database");

            String sql = "SELECT * FROM students";
            PreparedStatement ps = con.prepareStatement(sql);
            ResultSet rs = ps.executeQuery();

            System.out.println("ID\tName\tAge\tEmail");
            System.out.println("-----");
            while (rs.next()) {
                int id = rs.getInt("id");
                String name = rs.getString("name");
                int age = rs.getInt("age");
                String email = rs.getString("email");
                System.out.println(id + "\t" + name + "\t\t" + age + "\t" + email);
            }
        } catch (SQLException e) {
            System.out.println("Error: " + e);
        }
    }
}
```

```
//UpdateData.java
```

```
import java.sql.*;
import java.util.Scanner;

public class UpdateData {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/timr4";
        String user = "root";
        String password = "password";
        try {
            Connection con = DriverManager.getConnection(url, user, password);
            System.out.println("Connected to Database");

            Scanner sc = new Scanner(System.in);

            System.out.print("Enter student ID to update: ");
            int sid = sc.nextInt();
            sc.nextLine();

            System.out.print("Enter new name: ");
            String sname = sc.nextLine();

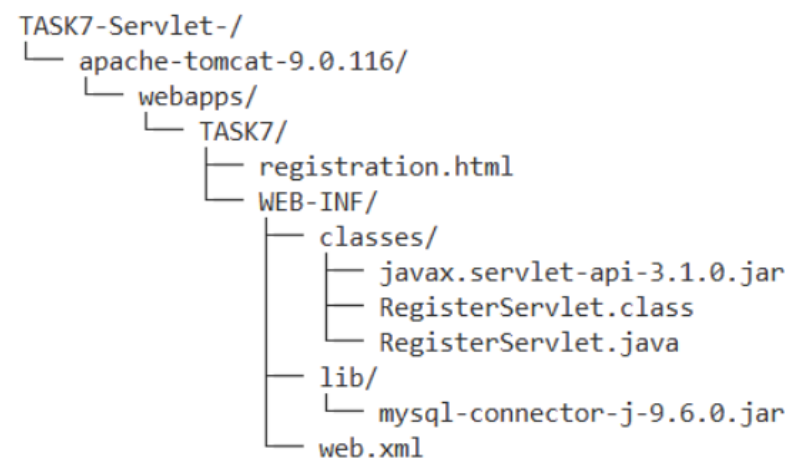
            System.out.print("Enter new age: ");
            int sage = sc.nextInt();
            sc.nextLine();

            System.out.print("Enter new email: ");
            String semail = sc.nextLine();

            String sql = "UPDATE students SET name=?, age=?, email=? WHERE id=?";
            PreparedStatement ps = con.prepareStatement(sql);
            ps.setString(1, sname);
            ps.setInt(2, sage);
            ps.setString(3, semail);
            ps.setInt(4, sid);

            int rowsAffected = ps.executeUpdate();
            if (rowsAffected > 0) {
                System.out.println("Record Updated Successfully.");
            } else {
                System.out.println("No record found with ID: " + sid);
            }
            sc.close();
        } catch (SQLException e) {
            System.out.println("Error: " + e);
        }
    }
}
```

```
1 #bookstore_xsd.xml
2 <?xml version="1.0"?>
3   <bookstore xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xsi:noNamespaceSchemaLocation="bookstore.xsd">
4
5     <book>
6       <title>Java Programming</title>
7       <author>James Gosling</author>
8       <price>500</price>
9     </book>
10    <book>
11      <title>Python Basics</title>
12      <author>Guido Van Rossum</author>
13      <price>450</price>
14    </book>
15    <book>
16      <title>Web Development</title>
17      <author>John Smith</author>
18      <price>550</price>
19    </book>
20
21    <book>
22      <title>Database Systems</title>
23      <author>Korth</author>
24      <price>600</price>
25    </book>
26
27    <book>
28      <title>Data Structures</title>
29      <author>Mark Allen</author>
30      <price>520</price>
31    </book>
32
33  </bookstore>
34 //bookstore_internal_dtd.xml
35 <?xml version="1.0"?>
36
37 <!DOCTYPE bookstore [
38   <!ELEMENT bookstore (book+)>
39   <!ELEMENT book (title, author, price)>
40   <!ELEMENT title (#PCDATA)>
41   <!ELEMENT author (#PCDATA)>
42   <!ELEMENT price (#PCDATA)>
43 ]>
44
45 <bookstore>
46
47   <book>
48     <title>Java Programming</title>
49     <author>James Gosling</author>
50     <price>500</price>
51   </book>
52
53   <book>
54     <title>Python Basics</title>
55     <author>Guido Van Rossum</author>
56     <price>450</price>
57   </book>
58
59   <book>
60     <title>Web Development</title>
61     <author>John Smith</author>
62     <price>550</price>
63   </book>
64
65   <book>
66     <title>Database Systems</title>
67     <author>Korth</author>
68     <price>600</price>
69   </book>
70
71   <book>
72     <title>Data Structures</title>
73     <author>Mark Allen</author>
74     <price>520</price>
75   </book>
76
77 </bookstore>
```



```
1  #registration.html
2  <!doctype html>
3  <html>
4      <head>
5          <title>User Registration</title>
6      </head>
7      <body>
8          <h2>User Registration</h2>
9
10         <form method="post" action="RegisterServlet">
11             <label>Username:</label><br />
12             <input type="text" name="name" required /><br /><br />
13
14             <label>Password:</label><br />
15             <input type="password" name="password" required /><br /><br />
16
17             <label>Email:</label><br />
18             <input type="email" name="email" required /><br /><br />
19
20             <label>Contact Number:</label><br />
21             <input type="text" name="contact" required /><br /><br />
22
23             <input type="submit" value="Register" />
24         </form>
25     </body>
26 </html>
27
28 #web.xml
29 <web-app>
30     <servlet>
31         <servlet-name>RegisterServlet</servlet-name>
32         <servlet-class>RegisterServlet</servlet-class>
33     </servlet>
34
35     <servlet-mapping>
36         <servlet-name>RegisterServlet</servlet-name>
37         <url-pattern>/RegisterServlet</url-pattern>
38     </servlet-mapping>
39
40 </web-app>
41 #RegisterServlet.java
42 import javax.servlet.*;
43 import javax.servlet.http.*;
44 import java.io.*;
45 import java.sql.*;
46
47 public class RegisterServlet extends HttpServlet {
48
49     protected void doPost(HttpServletRequest request, HttpServletResponse response)
50         throws ServletException, IOException {
51
52         response.setContentType("text/html");
53         PrintWriter pw = response.getWriter();
54
55         String name = request.getParameter("name");
56         String password = request.getParameter("password");
57         String email = request.getParameter("email");
58         String contact = request.getParameter("contact");
59
60         String url = "jdbc:mysql://localhost:3306/registration_DB_5R4";
61         String user = "root";
62         String pwd = "password";
63
64         try {
65             // ✅ Correct driver
66             Class.forName("com.mysql.cj.jdbc.Driver");
67
68             Connection con = DriverManager.getConnection(url, user, pwd);
69
70             // ✅ SQL matches form fields
71             String sql = "INSERT INTO students(name, password, email, contact) VALUES (?, ?, ?, ?)";
72             PreparedStatement ps = con.prepareStatement(sql);
73
74             // ✅ Use form data (NOT Scanner)
75             ps.setString(1, name);
76             ps.setString(2, password);
77             ps.setString(3, email);
```

```

78         ps.setString(4, contact);
79
80         int rows = ps.executeUpdate();
81
82         if (rows > 0) {
83             pw.println("<h2>Registration Successful!</h2>");
84         } else {
85             pw.println("<h2>Registration Failed</h2>");
86         }
87
88         con.close();
89
90     } catch (Exception e) {
91         pw.println("Error: " + e.getMessage());
92     }
93
94     pw.close();
95 }
96 }
97 #cookie.html
98 <!doctype html>
99 <html>
100     <head>
101         <title>Cookie Login</title>
102     </head>
103     <body>
104         <h2>Login using Cookies</h2>
105
106         <form action="login" method="post">
107             Username: <input type="text" name="username" required /><br /><br />
108             Password:
109             <input type="password" name="password" required /><br /><br />
110             <input type="submit" value="Login" />
111         </form>
112     </body>
113 </html>
114
115 #web.xml
116 <web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee"
117     version="3.1">
118
119     <servlet>
120         <servlet-name>cookieLogin</servlet-name>
121         <servlet-class>CookieLoginServlet</servlet-class>
122     </servlet>
123
124     <servlet-mapping>
125         <servlet-name>cookieLogin</servlet-name>
126         <url-pattern>/login</url-pattern>
127     </servlet-mapping>
128
129     <servlet>
130         <servlet-name>cookieRetrieve</servlet-name>
131         <servlet-class>CookieRetrieveServlet</servlet-class>
132     </servlet>
133
134     <servlet-mapping>
135         <servlet-name>cookieRetrieve</servlet-name>
136         <url-pattern>/retrieve</url-pattern>
137     </servlet-mapping>
138
139 </web-app>
140 #CookieLoginServlet.java
141 import java.io.*;
142 import javax.servlet.*;
143 import javax.servlet.http.*;
144
145 public class CookieLoginServlet extends HttpServlet {
146
147     protected void doPost(HttpServletRequest req, HttpServletResponse res)
148         throws IOException {
149
150         res.setContentType("text/html;charset=UTF-8");
151
152         PrintWriter out = res.getWriter();
153
154         String username = req.getParameter("username");

```





```

155     String password = req.getParameter("password");
156
157     // Validation
158     if (username == null || username.isEmpty() ||
159         password == null || password.isEmpty()) {
160
161         out.println("<h3 style='color:red;'>Invalid input! Please try again.</h3>");
162         return;
163     }
164
165     // Create cookies (INCLUDING password as per requirement)
166     Cookie userCookie = new Cookie("uname", username);
167     Cookie passCookie = new Cookie("pwd", password);
168
169     // Expiry
170     userCookie.setMaxAge(60 * 60 * 24); // 1 day
171     passCookie.setMaxAge(2 * 60); // 2 minutes
172
173     // Basic security improvement (still allowed)
174     userCookie.setHttpOnly(true);
175     passCookie.setHttpOnly(true);
176
177     res.addCookie(userCookie);
178     res.addCookie(passCookie);
179
180     out.println("<h2>Cookies Created Successfully</h2>");
181     out.println("<a href='retrieve'>Retrieve Cookie Details</a>");
182 }
183 }

```

```

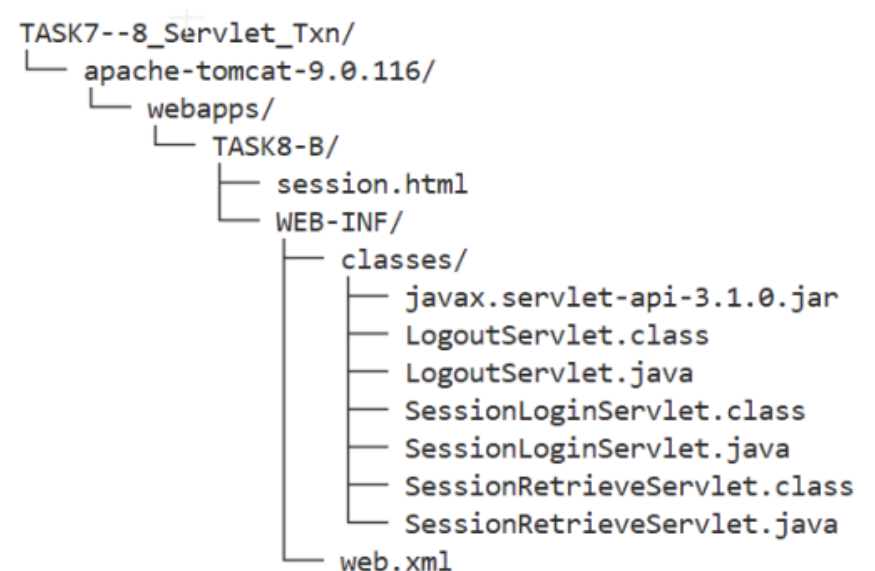
184 #CookieRetrieveServlet.java
185 import java.io.*;
186 import javax.servlet.*;
187 import javax.servlet.http.*;
188
189 public class CookieRetrieveServlet extends HttpServlet {
190
191     protected void doGet(HttpServletRequest req, HttpServletResponse res)
192         throws IOException {
193
194         res.setContentType("text/html;charset=UTF-8");
195
196         PrintWriter out = res.getWriter();
197
198         Cookie[] cookies = req.getCookies();
199
200         String username = null;
201         String password = null;
202
203         if (cookies != null) {
204             for (Cookie c : cookies) {
205
206                 if ("uname".equals(c.getName())) {
207                     username = c.getValue();
208                 }
209
210                 if ("pwd".equals(c.getName())) {
211                     password = c.getValue();
212                 }
213             }
214         }
215
216         if (username != null && password != null) {
217             out.println("<h2>Welcome " + username + "</h2>");
218             out.println("<h3>Password: " + password + "</h3>");
219         } else {
220             out.println("<h3>No cookies found. Please login first.</h3>");
221         }
222     }
223 }

```

```

224 #session.html
225 <!doctype html>
226 <html>
227     <head>
228         <title>Session Login</title>
229     </head>
230     <body>
231         <h2>Login using Session</h2>

```





```

232
233     <form action="login1" method="post">
234         Username: <input type="text" name="username" required /><br /><br />
235         Password:
236         <input type="password" name="password" required /><br /><br />
237         <input type="submit" value="Login" />
238     </form>
239 </body>
240 </html>
241
242 #web.xml
243 <web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">
244
245     <servlet>
246         <servlet-name>sessionLogin</servlet-name>
247         <servlet-class>SessionLoginServlet</servlet-class>
248     </servlet>
249
250     <servlet-mapping>
251         <servlet-name>sessionLogin</servlet-name>
252         <url-pattern>/login1</url-pattern>
253     </servlet-mapping>
254
255     <servlet>
256         <servlet-name>sessionRetrieve</servlet-name>
257         <servlet-class>SessionRetrieveServlet</servlet-class>
258     </servlet>
259
260     <servlet-mapping>
261         <servlet-name>sessionRetrieve</servlet-name>
262         <url-pattern>/retrieve1</url-pattern>
263     </servlet-mapping>
264
265     <servlet>
266         <servlet-name>logout</servlet-name>
267         <servlet-class>LogoutServlet</servlet-class>
268     </servlet>
269
270     <servlet-mapping>
271         <servlet-name>logout</servlet-name>
272         <url-pattern>/LogoutServlet</url-pattern>
273     </servlet-mapping>
274 </web-app>
275 #SessionLoginServlet.java
276 import java.io.*;
277 import javax.servlet.*;
278 import javax.servlet.http.*;
279
280 public class SessionLoginServlet extends HttpServlet {
281
282     protected void doPost(HttpServletRequest req, HttpServletResponse res)
283         throws IOException {
284
285         res.setContentType("text/html;charset=UTF-8");
286
287         PrintWriter out = res.getWriter();
288
289         String username = req.getParameter("username");
290         String password = req.getParameter("password");
291
292         if (username == null || username.isEmpty()) {
293             out.println("<h3 style='color:red;'>Invalid Login!</h3>");
294             return;
295         }
296
297         HttpSession session = req.getSession();
298
299         session.setAttribute("uname", username);
300         session.setAttribute("pwd", password); // 🔥 ADDED
301
302         session.setMaxInactiveInterval(30 * 60);
303
304         out.println("<h2>Login Successful (Session Created)</h2>");
305         out.println("<a href='retrieve1'>Retrieve Session Details</a>");
306     }
307 }
308

```



```

309 #SessionRetrieveServlet.java
310 import java.io.*;
311 import javax.servlet.*;
312 import javax.servlet.http.*;
313
314 public class SessionRetrieveServlet extends HttpServlet {
315
316     protected void doGet(HttpServletRequest req, HttpServletResponse res)
317         throws IOException {
318
319         res.setContentType("text/html;charset=UTF-8");
320
321         PrintWriter out = res.getWriter();
322
323         HttpSession session = req.getSession(false);
324
325         if (session != null) {
326
327             String user = (String) session.getAttribute("uname");
328             String pass = (String) session.getAttribute("pwd"); // 🔥 ADD
329
330             if (user != null && pass != null) {
331                 out.println("<h2>Welcome " + user + "</h2>");
332                 out.println("<h3>Password: " + pass + "</h3>");
333                 out.println("<a href='LogoutServlet'>Logout</a>");
334             } else {
335                 out.println("<h3>Session exists but data missing</h3>");
336             }
337
338         } else {
339             out.println("<h3>No session found. Please login first.</h3>");
340         }
341     }
342 }
343
344 #LogoutServlet.java
345 import java.io.*;
346 import javax.servlet.*;
347 import javax.servlet.http.*;
348
349 public class LogoutServlet extends HttpServlet {
350
351     protected void doGet(HttpServletRequest req, HttpServletResponse res)
352         throws IOException {
353
354         res.setContentType("text/html;charset=UTF-8");
355
356         PrintWriter out = res.getWriter();
357
358         HttpSession session = req.getSession(false);
359
360         if (session != null) {
361             session.invalidate(); // 🔥 destroy session!!!!!!
362         }
363
364         out.println("<h2>Logged out successfully</h2>");
365         out.println("<a href='session.html'>Login Again</a>");
366     }
367 }

```

```

1  #event.js
2  /* import EventEmitter from "events";
3  const myEmitter = new EventEmitter();
4
5  myEmitter.on("greet", (name) => {
6      console.log(`Hello ${name}`);
7  });
8
9  myEmitter.emit("greet", "Tim");
10 */
11
12 import EventEmitter from "events";
13
14 const myEE = new EventEmitter();
15
16 myEE.on("add", (a, b) => {
17     console.log(`a + b = ${a + b}`);
18 });
19
20 myEE.emit("add", 10, 20);
21
22 #os.js
23 import os from "os";
24 let currentOS = {
25     name: os.type(),
26     architecture: os.arch(),
27     platform: os.platform(),
28     release: os.release(),
29     version: os.version(),
30     freemem: (os.freemem() / 1024 ** 3).toFixed(2) + ` GB`,
31 };
32 console.log("Details of OS");
33 console.log(currentOS);
34
35 #path.js
36 /* import path from "path";
37 import { fileURLToPath } from "url";
38
39 const __filename = fileURLToPath(import.meta.url);
40 const __dirname = path.dirname(__filename);
41
42 console.log("File Name:", path.basename(__filename));
43 console.log("Full Path:", __filename);
44 console.log("Location:", __dirname);
45
46
47 */
48 import path from "path";
49
50 const myFILEpath = "C:\\Users\\Anurodh Saxena\\Documents\\MGIT\\atten.html";
51 console.log("file name: " + path.basename(myFILEpath));
52 console.log("ext name: " + path.extname(myFILEpath));
53
54 #server.js
55 /*
56
57 import http from "http";
58
59 http.createServer((req, res) => {
60     res.end("Hello World");
61 }).listen(3000);
62
63 console.log("Server running on port 3000");
64
65 */
66
67 import http from "http";
68
69 const port = 3000;
70
71 const server = http.createServer((req, res) => {
72     res.end("Hi praise the lord jesus");
73 });
74
75 server.listen(port, () => {
76     console.log(`server running on port ${port}`);
77 });

```

```

1  const e = require("express");
2  const m = require("mysql2");
3
4  const a = e();
5  a.use(e.json());
6
7  const p = m
8    .createPool({
9      host: "127.0.0.1",
10     user: "root",
11     password: "password",
12     database: "college_db_5R4",
13     port: 3306,
14   })
15   .promise();
16
17  // READ
18  a.get("/studentdetails_5R4", async (q, s) => {
19    try {
20      const [r] = await p.query("SELECT * FROM studentdetails_5R4");
21      s.json(r);
22    } catch (er) {
23      s.status(500).json({ message: er.message });
24    }
25  });
26
27  // CREATE
28  a.post("/studentdetails_5R4", async (q, s) => {
29    const { name, course } = q.body;
30    try {
31      const [r] = await p.execute(
32        "INSERT INTO studentdetails_5R4 (name,course) VALUES (?,?)",
33        [name, course],
34      );
35      s.status(201).json({ id: r.insertId, name, course });
36    } catch (er) {
37      s.status(500).json({ error: er.message });
38    }
39  });
40
41  // UPDATE
42  a.put("/studentdetails_5R4/:id", async (q, s) => {
43    const { id } = q.params;
44    const { name, course } = q.body;
45    try {
46      const [r] = await p.execute(
47        "UPDATE studentdetails_5R4 SET name=?,course=? WHERE id=?",
48        [name, course, id],
49      );
50      if (r.affectedRows === 0)
51        return s.status(404).json({ message: "Student not found" });
52
53      s.json({ message: "Student updated successfully", id, name, course });
54    } catch (er) {
55      s.status(500).json({ error: er.message });
56    }
57  });
58
59  // DELETE
60  a.delete("/studentdetails_5R4/:id", async (q, s) => {
61    const { id } = q.params;
62    try {
63      const [r] = await p.execute(
64        "DELETE FROM studentdetails_5R4 WHERE id=?",
65        [id],
66      );
67      if (r.affectedRows === 0)
68        return s.status(404).json({ message: "Student not found" });
69
70      s.json({ message: `Student with ID ${id} deleted successfully` });
71    } catch (er) {
72      s.status(500).json({ error: er.message });
73    }
74  });
75
76  a.listen(3000, () => console.log("Server running on port 3000"));
77

```

```

1  const e = require("express");
2  const m = require("mysql2");
3  const j = require("jsonwebtoken");
4
5  const app = e();
6  app.use(e.json());
7
8  const K = "mgit_secret_key";
9
10 const db = m
11     .createPool({
12         host: "127.0.0.1",
13         user: "root",
14         password: "password",
15         database: "college_db_5r4",
16         port: 3306,
17     })
18     .promise();
19
20 // middleware
21 const auth = (req, res, next) => {
22     let t = req.headers["authorization"];
23     t = t && t.split(" ")[1];
24     if (!t)
25         return res
26             .status(401)
27             .json({ message: "Access Denied: No Token Provided" });
28
29     j.verify(t, K, (err, u) => {
30         if (err)
31             return res
32                 .status(403)
33                 .json({ message: "Invalid or Expired Token" });
34         req.user = u;
35         next();
36     });
37 };
38
39 // login
40 app.post("/login", (req, res) => {
41     let { username, password } = req.body;
42
43     if (username === "admin" && password === "password123") {
44         let t = j.sign({ username }, K, { expiresIn: "7d" });
45         return res.json({ token: t });
46     }
47
48     res.status(401).json({ message: "Invalid Credentials" });
49 });
50
51 // GET OR READ
52 app.get("/s_details", auth, async (req, res) => {
53     try {
54         let [r] = await db.query("SELECT * FROM s_details");
55         res.json(r);
56     } catch (e) {
57         res.status(500).json({ message: e.message });
58     }
59 });
60
61 // POST
62 app.post("/s_details", auth, async (req, res) => {
63     let { name, course } = req.body;
64
65     try {
66         let [r] = await db.execute(
67             "INSERT INTO s_details (name, course) VALUES (?, ?)",
68             [name, course],
69         );
70
71         res.status(201).json({ id: r.insertId, name, course });
72     } catch (e) {
73         res.status(500).json({ error: e.message });
74     }
75 });
76
77 // PUT

```

```

78 app.put("/s_details/:id", auth, (req, res) => {
79     let { id } = req.params;
80     let { name, course } = req.body;
81
82     try {
83         let [r] = await db.execute(
84             "UPDATE s_details SET name=?, course=? WHERE id=?",
85             [name, course, id],
86         );
87
88         if (!r.affectedRows)
89             return res.status(404).json({ message: "Not found" });
90
91         res.json({ message: "Updated successfully", id });
92     } catch (e) {
93         res.status(500).json({ error: e.message });
94     }
95 });
96
97 // DELETE
98 app.delete("/s_details/:id", auth, async (req, res) => {
99     let { id } = req.params;
100
101     try {
102         let [r] = await db.execute("DELETE FROM s_details WHERE id=?", [id]);
103
104         if (!r.affectedRows)
105             return res.status(404).json({ message: "Not found" });
106
107         res.json({ message: `Student ${id} deleted` });
108     } catch (e) {
109         res.status(500).json({ error: e.message });
110     }
111 });
112
113 app.listen(3000, () => console.log("Server running on port 3000"));
114
115 #####App.js
116 import React from "react";
117 import { BrowserRouter as Router, Routes, Route, Link } from "react-router-dom";
118
119 import Home from "./pages/Home";
120 import Register from "./pages/Register";
121 import Login from "./pages/Login";
122 import About from "./pages/About";
123 import Contact from "./pages/Contact";
124
125 function App() {
126     return (
127         <Router>
128             <nav style={{ padding: "10px", background: "black" }}>
129                 <Link to="/" style={linkStyle}>
130                     Home
131                 </Link>
132                 <Link to="/register" style={linkStyle}>
133                     Register
134                 </Link>
135                 <Link to="/login" style={linkStyle}>
136                     Login
137                 </Link>
138                 <Link to="/about" style={linkStyle}>
139                     About
140                 </Link>
141                 <Link to="/contact" style={linkStyle}>
142                     Contact
143                 </Link>
144             </nav>
145
146             <Routes>
147                 <Route path="/" element={<Home />} />
148                 <Route path="/register" element={<Register />} />
149                 <Route path="/login" element={<Login />} />
150                 <Route path="/about" element={<About />} />
151                 <Route path="/contact" element={<Contact />} />
152             </Routes>
153         </Router>
154     );

```

```
155 }
156
157 const linkStyle = {
158   color: "white",
159   margin: "10px",
160   textDecoration: "none",
161 };
162
163 export default App;
164 #Home.js
165 export default function Home() {
166   return (
167     <div>
168       <h1>Home Page</h1>
169       <p>Welcome to Student Management System</p>
170     </div>
171   );
172 }
173
174 #Register.js
175 export default function Register() {
176   return (
177     <div>
178       <h2>Register</h2>
179
180       <input placeholder="Name" />
181       <br />
182       <input placeholder="Email" />
183       <br />
184       <input type="password" placeholder="Password" />
185       <br />
186
187       <button>Register</button>
188     </div>
189   );
190 }
191
192 #Login.js
193 export default function Login() {
194   return (
195     <div>
196       <h2>Login</h2>
197
198       <input placeholder="Email" />
199       <br />
200       <input type="password" placeholder="Password" />
201       <br />
202
203       <button>Login</button>
204     </div>
205   );
206 }
207
208 #About.js
209 export default function About() {
210   return <h2>About Page</h2>;
211 }
212
213 #Contact.js
214 export default function Contact() {
215   return <h2>Contact Page</h2>;
216 }
217
218
219
```